date

Heap Sort

Heap sort is a comparison based sort built on binary heaps data structure.

Binary heap data structure:

we will focus on "Max heap"

A max heap is a complete binary tree where every node satisfies:

$$Parent[i] \geq child[i]$$

* Also left child should exist before right child.

When our elements are in one-indexed array A:

root = $A[1]$

Left child = $A[2i]$

Right child = $A[2i+1]$

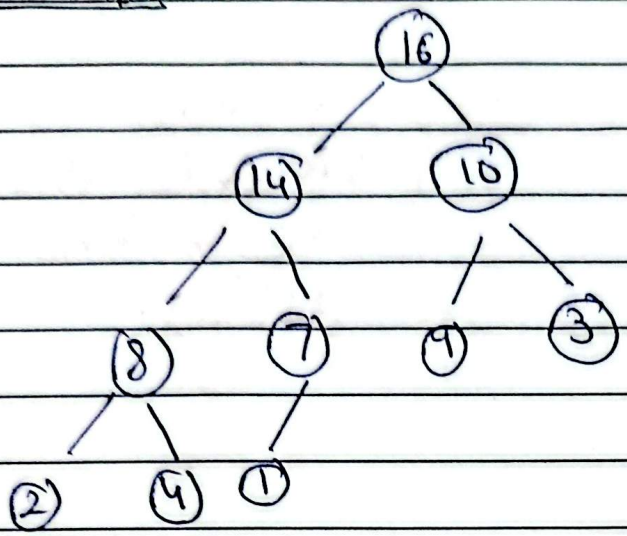
Parent of child = $A[\lfloor i/2 \rfloor]$

BHATTI

CamScanner

ay/date

Example



Max heap.

In array:

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

e.g child of 14

$= A[2*2], A[2*2+1]$

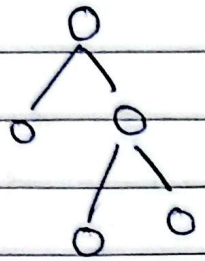
$= 8, 7.$

for array: Just read left to right, level by level

BHATTI

//date

(2)



✗ Not a heap.
(left child should be before right child).

Now we have understood the core understanding of a Heap specifically Max-Heap.

(Min heap is basically same but parent[i] ≤ child[i].)

BHATTI

CamScanner

te

Heap Sort:

There are 3 steps / functions for Heapsort.

1. Heapify (Max)
2. Build Max Heap
3. HeapSort

① Heapify:

Pseudocode

```
MAX-HEAPIFY (A, i, heap_size)
    left = 2 * i
    right = 2 * i + 1

    largest = i

    if left < heapsize && A[left] > A[largest]
        largest = left;

    if right < heap_size && A[right] > A[largest
        Largest = right.

    if largest != i
        exchange A[i] with A[largest]
        MAX-Heapify (A, largest, heap size)
```

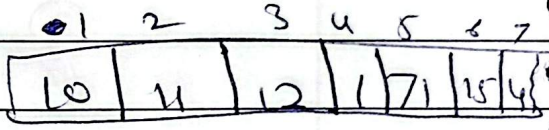
BHATTI

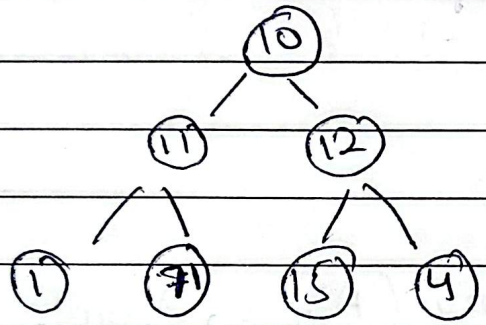
CamScanner

This is basically a recursive function that keeps on going down the tree.

(Complexity: O(log n))

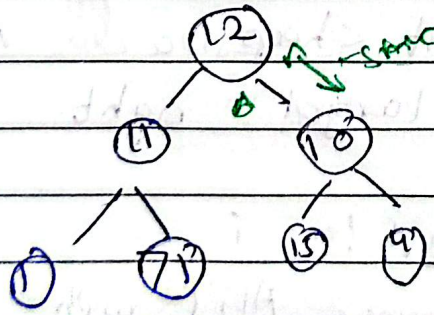
e.g lets consider.





consider we run heapify on root node i.e 1

now 12 is A[largest] so largest = 3, & exchang.



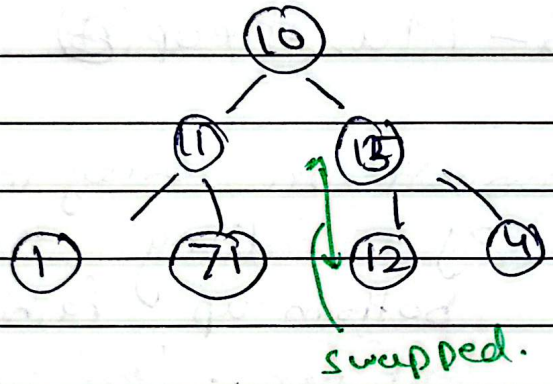
BHATTI

te

now again recursive call to max_heapify (A, 3, 7)

now largest = 3, so i = 3 in this run.

again call:



now largest = 6

called again

Max_heapify (A, 6, 7)

but now left = 12

& right = 13

12 > 7 & 13 > 7

so recursion ends

& Heapify done.

BHATTI

CamScanner

date

But still we can see that our tree is not a heap. It got fixed on one branch but not other.

(2) Build-Max-Heap

This converts an array into a max heap by calling MAX-Heapify in a bottom up fashion starting from last non-leaf node ($\lfloor n/2 \rfloor$) down to 1

```
BUILD-MAX-HEAP(A)
heap_size = length(A)
for i ← ⌊n/2⌋ down to 1
    MAX_HEAPIFY(A, i, heapsize).
```

complexity = $O(n) * O(\log n)$

$= O(n \log n)$

$O(n)$ ↓ loop

$O(\log n)$ ↓ MaxHeapify.

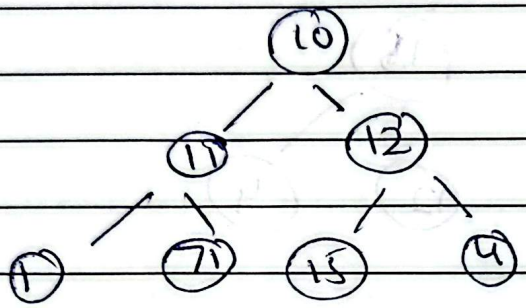
BHATTI

late

Example:

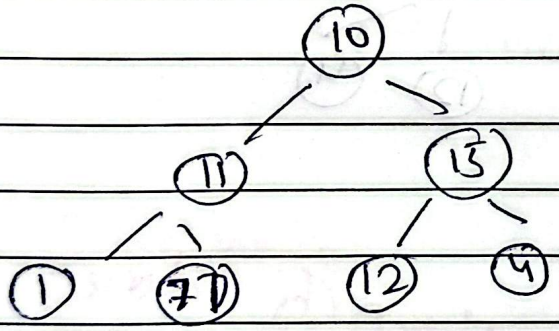
1	2	3	4	5	6	7
10	11	12	1	71	15	4

n = 7

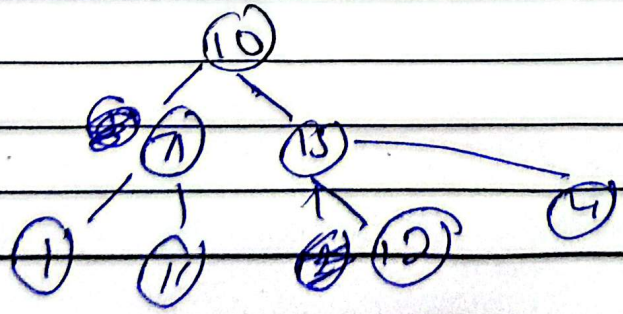


start with $\lfloor \frac{n}{2} \rfloor = 3$.

so 12



next 3-1 = 2 so 11.

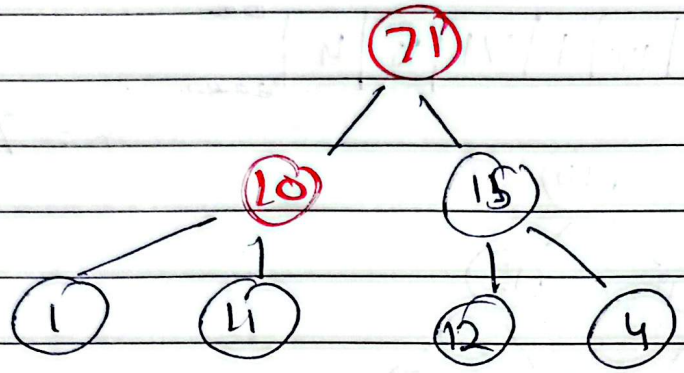


BHATTI

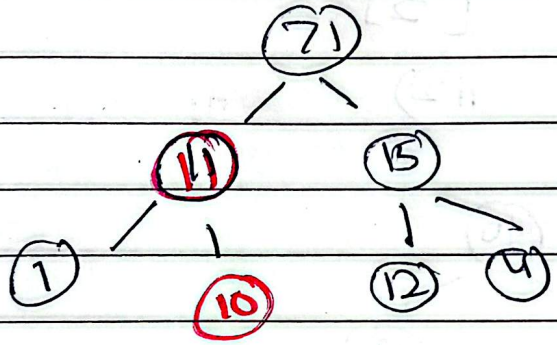
CamScanner

ay/date

now 2-1 = 1, so (10)



now again recurse down.



Build heap fully done.

BHATTI

CamScanner

ate

Now we are getting a true Max_Heap BUT it is not sorted.

③ HeapSort:

```
HEAP-SORT (A)

    BUILD-MAX-HEAP(A)
    heap-size = length (A)

    for i = length(A) down to 1
        exchange A[1] with A[i]
        heap-size = heap-size -1
        MAX-HEAPIFY (A, 1, heap-size).
```

Complexity:

$O(n \log n) + O(n) * O(\log n)$.

$= O(n \log n)$

BHATTI

CamScanner

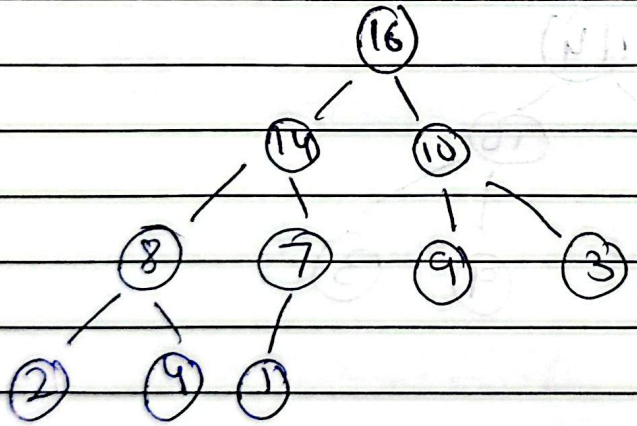
y/date

Understanding the algo:

In each iteration we are setting the ~~largest~~ highest value in correct place. Hence we start to reduce size of Heap too because the last element everytime can be ignored. Then we call max_heapify & bring the next highest to the top.

Example:

16	14	10	8	7	9	3	2	4	1



1) Build Heap first

Already Heap. so no change.

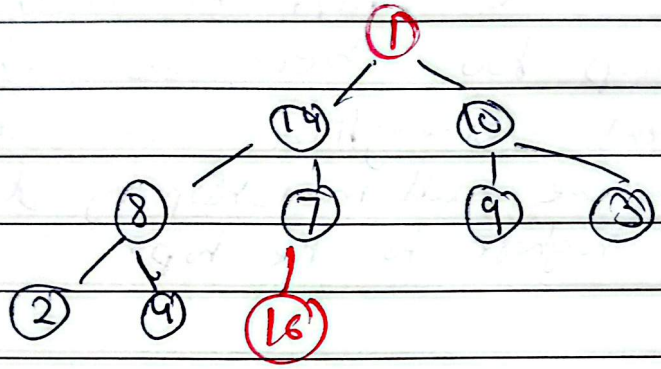
BHATTI

CamScanner

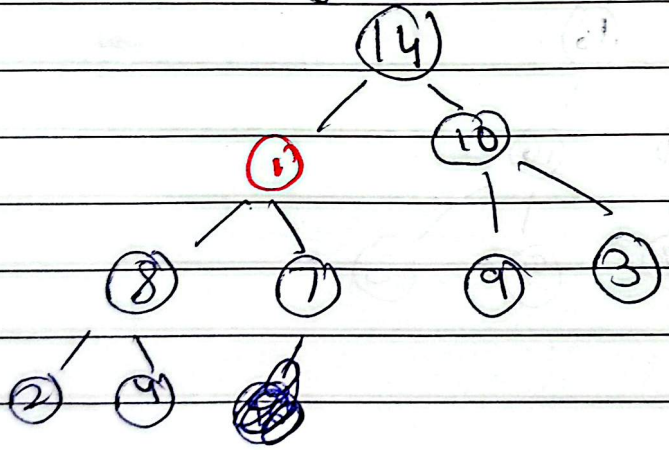
ay/date

now comy the actual loop.

1) exchange A[1], root with last element.



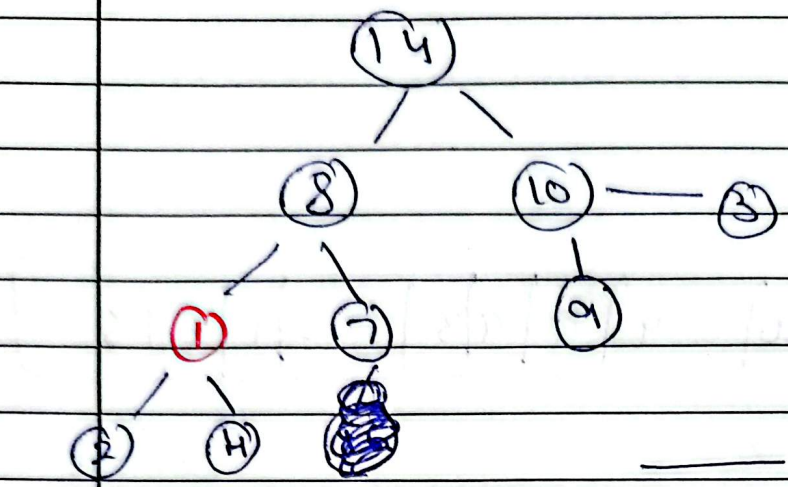
now ~~heap~~ Max-heap from start.
BUT ignore last element.

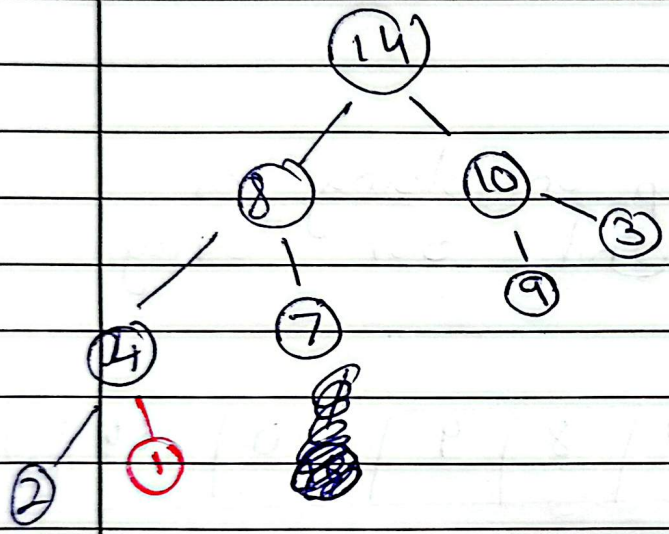


BHATTI

CamScanner

day/date



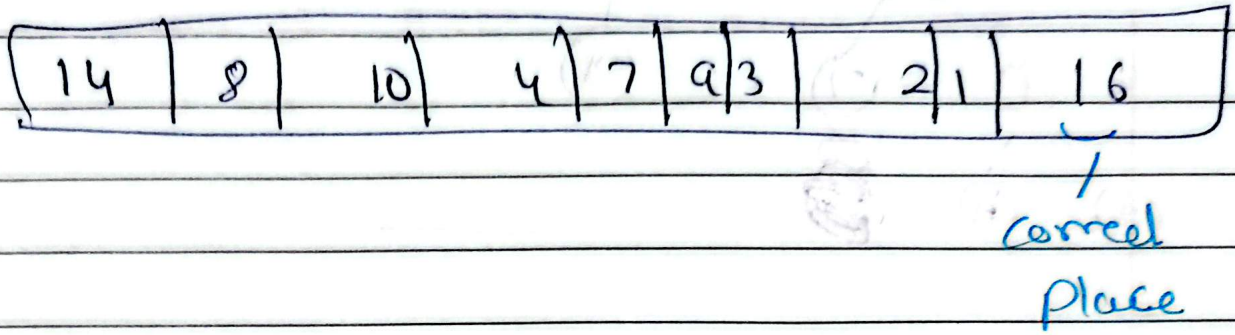


Max-Heapify ends.

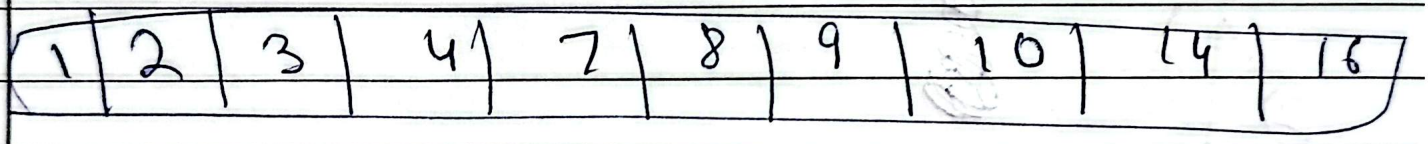
BHATTI

CamScanner

now array:



now keeep on doing this & at end our array looks like

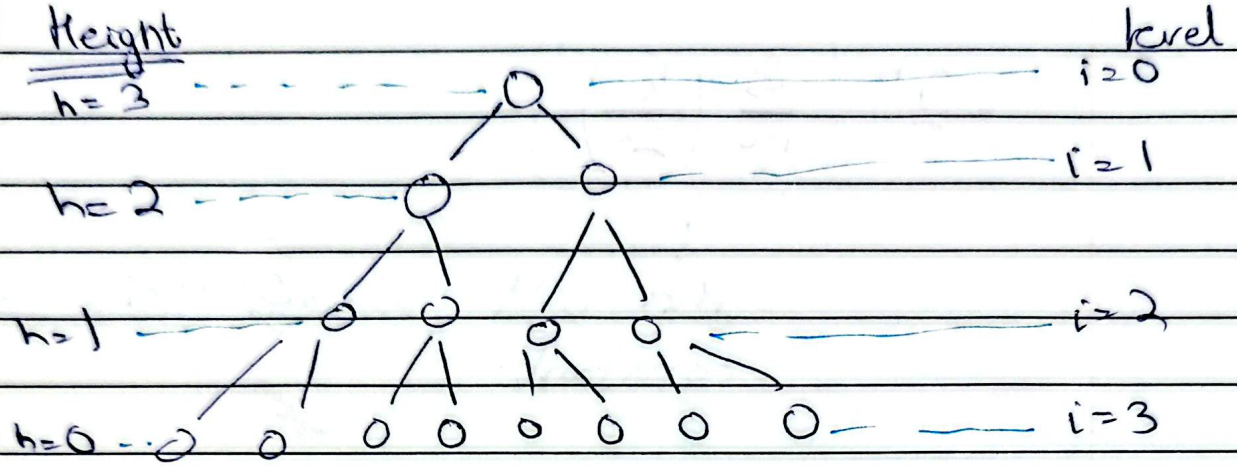


BHATTI

/date

Careful complexity analysis of HeapSort

(Best case)



$$= \sum_{i=0}^{h} n_i * h_i$$

n_i → nodes on each leave

h_i → max heapify work. (go down full height).

$$= \sum_{i=0}^{h} 2^i * (h-i)$$

removed n from equation.

$$= \sum_{i=0}^{h} \frac{2^h}{2^{h-i}} * (h-i)$$

$\# \; 2^i = \frac{2^h}{2^{h-i}}$

$$= 2^h \sum_{i=0}^{h} \left(\frac{h-i}{2^{h-i}} \right)$$

BHATTI

y/date

lets take $h-i = k$

$= 2^h \left[\sum_{i=0}^{h} \left(\frac{k}{2^k} \right) \right]$

↓

this will always yield some constant.

so our complexity $\leq 2^h$

$h = \log n$ # of height

so $\leq 2^{\log_2 n}$

$\leq \Theta(n)$.

Best case: $\Theta(n)$.

BHATTI

CamScanner